Module 6
File System Interface

File – An Introduction

A file is a named collection of related information that is recorded on secondary storage. File represent data and programs.

A file system is a structure that organizes and manages files on a storage device, such as a hard drive, solid state drive (SSD), or USB flash drive. It defines how data is stored, accessed, and organized on the storage device.

The operating system abstracts from the physical properties of its storage devices to define a logical storage unit called the **file**

File attributes

- **Name.** The symbolic file name is the only information kept in human readable form.
- **Identifier**. This unique tag, usually a number, identifies the file within the file system
- **Type.**
- **Location.** This information is a pointer to a device and to the location of the file on that device.
- **Size.** The current size of the file (in bytes, words, or blocks)
- **Protection.** Access-control information determines who can do reading, writing, executing, and so on.
- **Timestamps and user identification.** This information may be kept for *creation*, *last modification*, *and last use*. These data can be *useful for protection*, *security*, *and usage monitoring*.

File operations

The seven basic operations comprise the minimal set of required file operations.

1. Creating a file
2. Opening a file
3. Writing a file
4. Reading a file
5. Repositioning within a file (seek)
6. Deleting a file
7. Truncating a file

File types

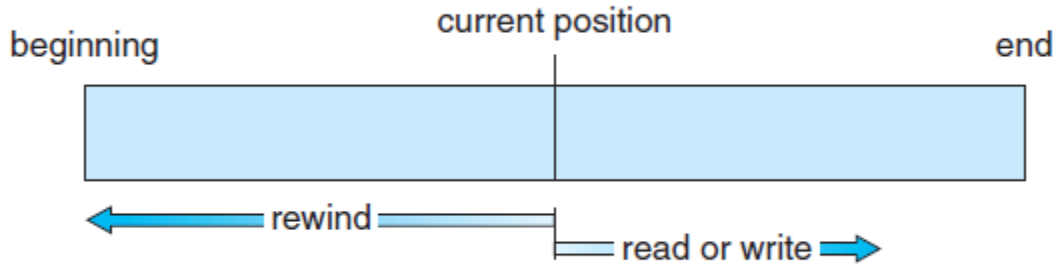
file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, perl, asm	source code in various languages
batch	bat, sh	commands to the command interpreter
markup	xml, html, tex	textual data, documents
word processor	xml, rtf, docx	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	gif, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	rar, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, mp3, mp4, avi	binary file containing audio or A/V information

File access methods

1. Sequential access

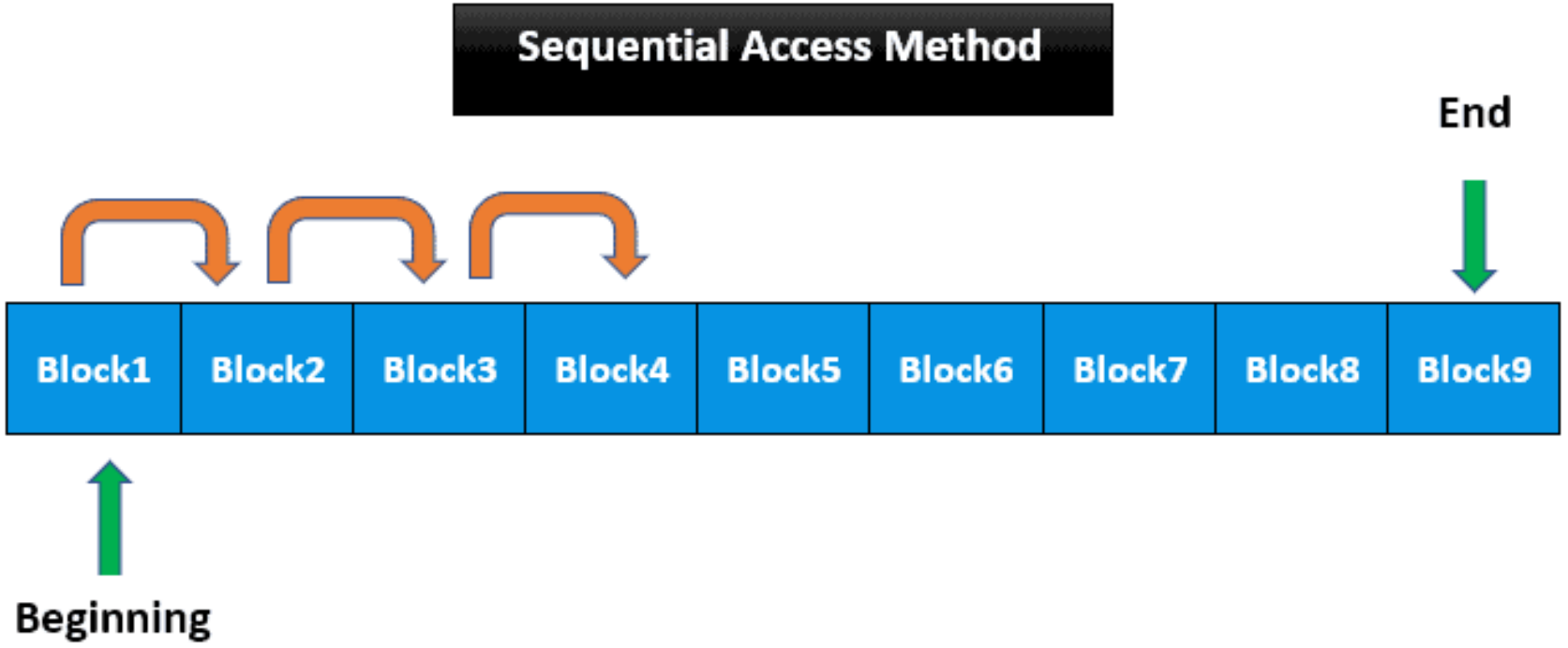
Information in the file is processed in order, one record after the other. This mode of access is by far the most common; for example, editors and compilers usually access files in this fashion.

A read operation—read next()—reads the next portion of the file and automatically advances a file pointer. The write operation—write next()—appends to the end of the file and advances to the end of the newly written material



File access methods

1. Sequential access



File access methods

2. Direct access (Relative access)

The direct access method allow random access to any file block

Here, a file is made up of fixed-length logical records that allow programs to read and write records rapidly in no particular order. There are no restrictions on the order of reading or writing for a direct-access file.

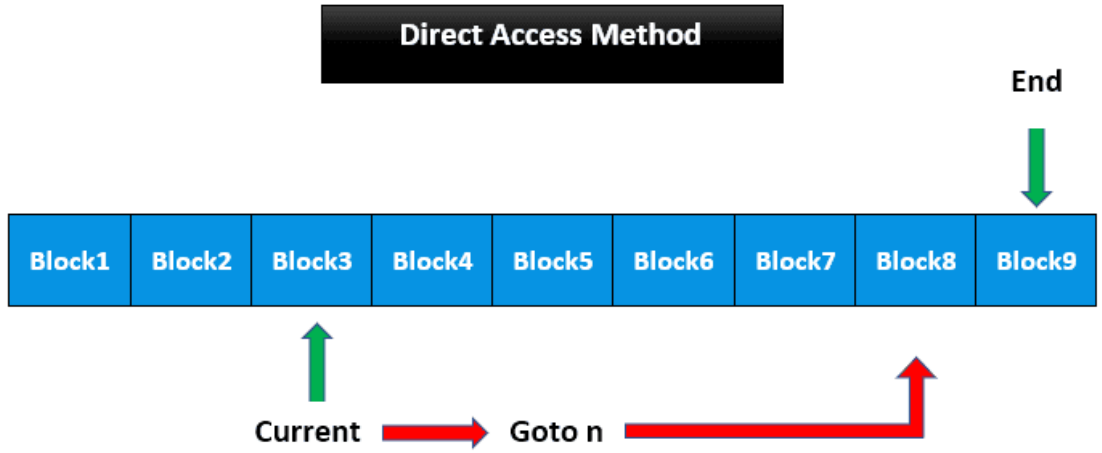
The direct access mechanism requires the OS to perform some additional tasks but eventually leads to much faster retrieval of records as compared to sequential access.

Direct-access files are of great use for immediate access to large amounts of information. Databases are often of this type.

File access methods

2. Direct access (Relative access)

- For the direct-access method, the file operations must be modified to include the block number as a parameter. For example, read(n), where n is the block number.
- The block number provided by the user to the operating system is normally a relative block number. A relative block number is an index relative to the beginning of the file.
- Not all operating systems support both sequential and direct access for files. Some systems allow only sequential file access; others allow only direct access.



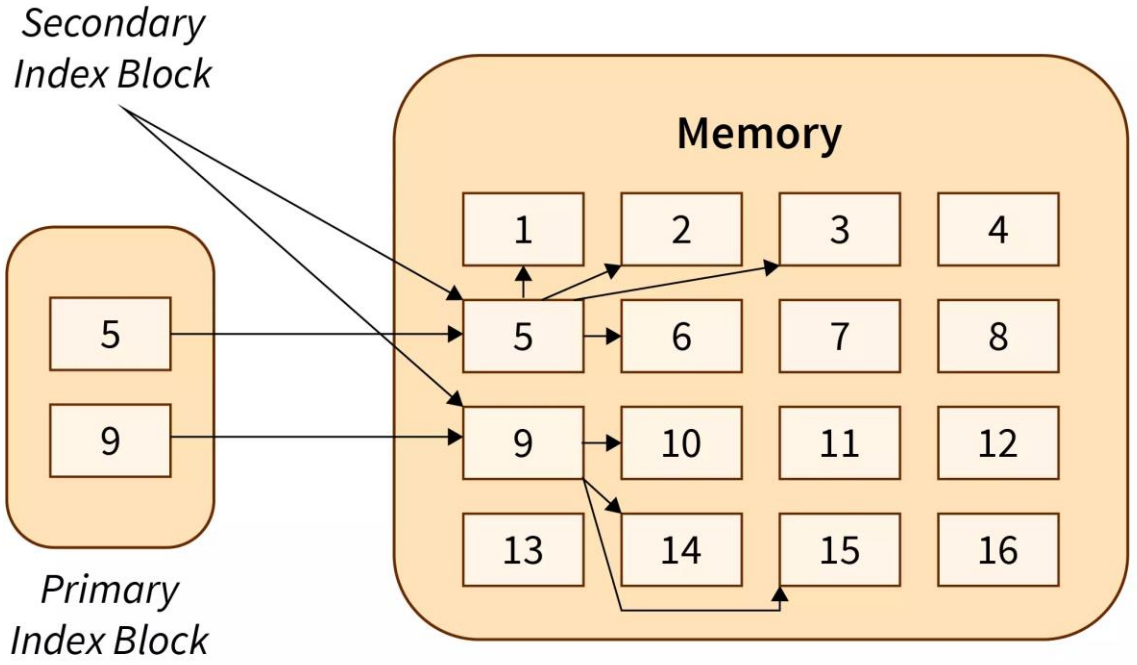
File access methods

3. Indexed Sequential Access

Indexed file access is a method that incorporates the benefits of both sequential and direct file access.

This method involves creating an index file that maps logical keys or data elements to their corresponding physical addresses within the file.

Moreover, the system stores the index separately from the data file, enabling quick access to locate the desired data.



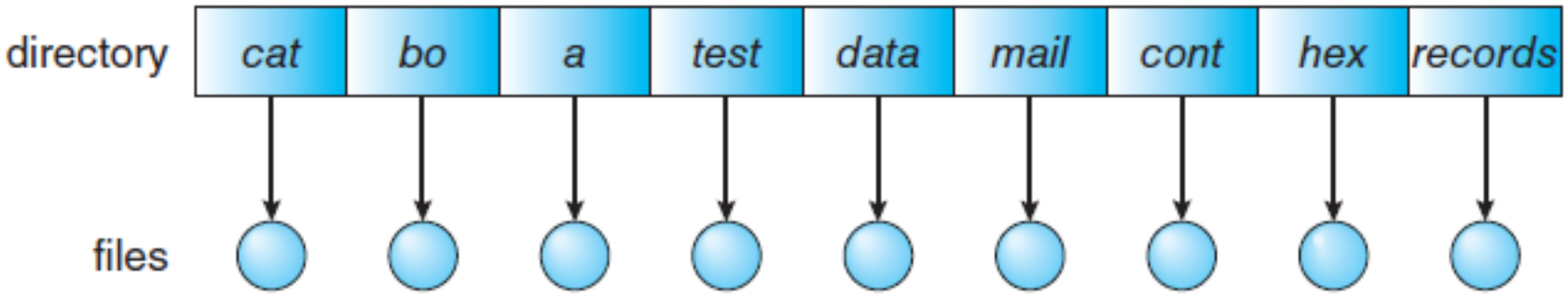
Directory Structure

The operations that are to be performed on a directory:

- Search a file
- Create and add a file in the directory
- Remove a file from the directory
- List the files in the directory
- Rename a file
- Traverse the file system

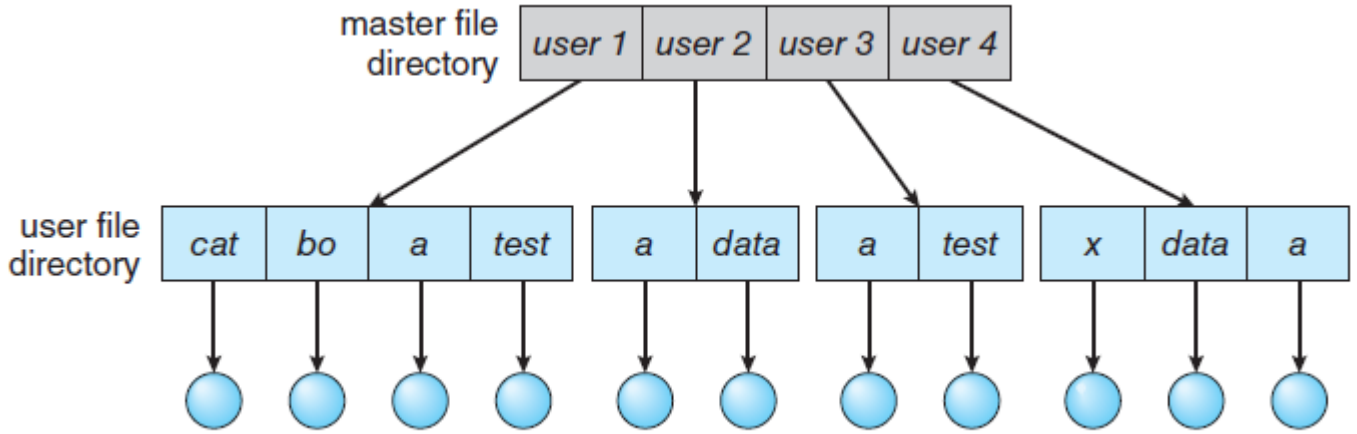
Directory Structure - Single-Level Directory

- The simplest directory structure is the single-level directory.
- All files are contained in the same directory
- Since all files are in the same directory, they must have unique names.



Directory Structure – Two-Level Directory

- Create a separate directory for each user.
- In the two-level directory structure, each user has his own user file directory (UFD).
- When a user job starts or a user logs in, the system's master file directory (MFD) is searched. The MFD is indexed by user name or account number, and each entry points to the UFD for that user



Directory Structure – Two-Level Directory

- When a user refers to a particular file, only his own UFD is searched
- To create a file for a user, the operating system searches only that user's UFD to ascertain whether another file of that name exists
- To delete a file, the operating system confines its search to the local UFD
- A user name and a file name define a path name and every file in the system has a path name.

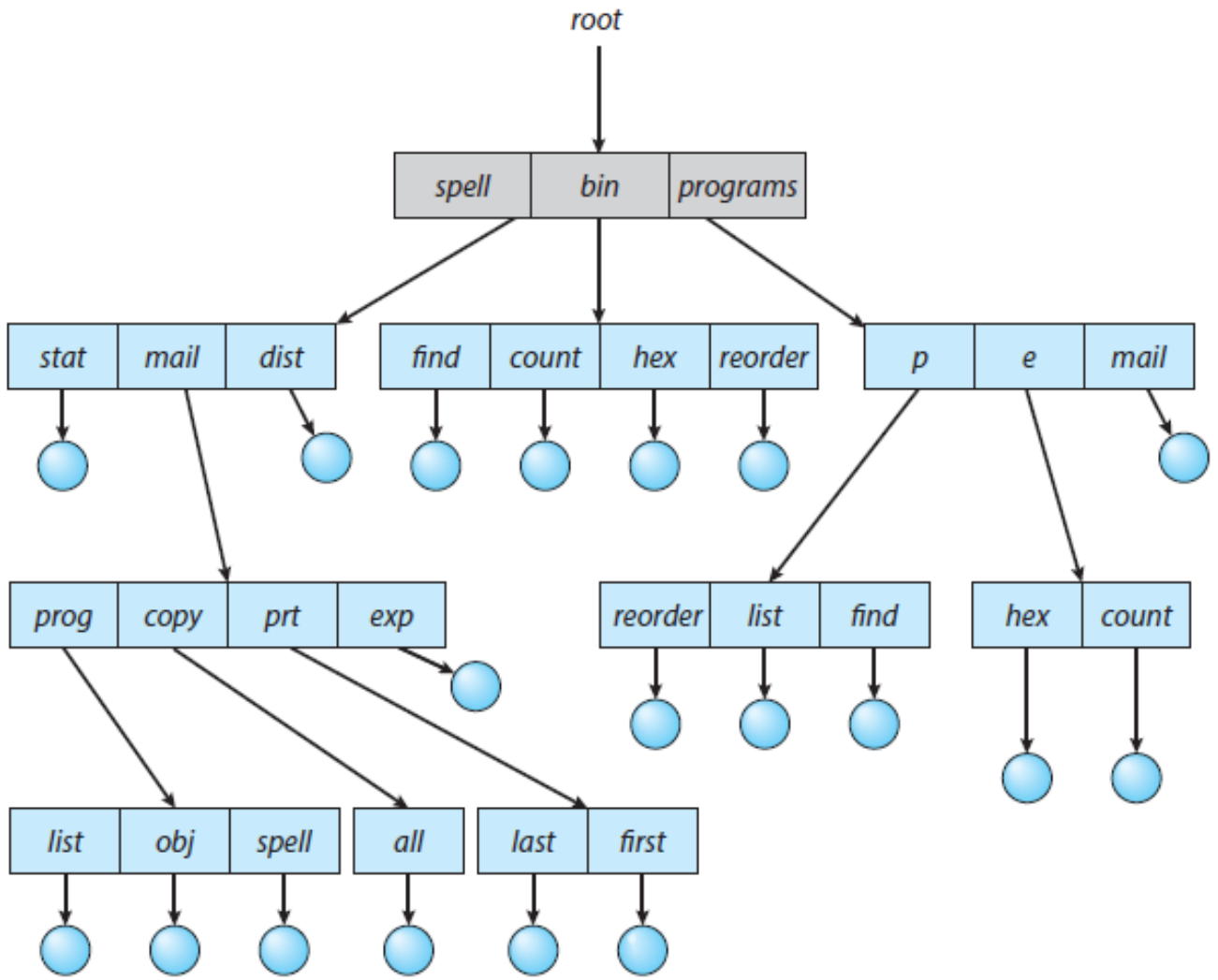
Advantage: solves the name-collision problem
Disadvantage: Sharing of files is not possible

Directory Structure – Tree-Structured Directories

- Extend the two-level directory structure to a tree of arbitrary height
- This generalization allows users to create their own subdirectories and to organize their files accordingly
- The tree has a root directory, and every file in the system has a unique path name.
- A directory (or subdirectory) contains a set of files or subdirectories
- All directories have the same internal format. One bit in each directory entry defines the entry as a file (0) or as a subdirectory (1).
- Special system calls are used to create and delete directories

- To change directories, a system call could be provided that takes a directory name as a parameter and uses it to redefine the current directory

Directory Structure – Tree-Structured Directories



Directory Structure – Tree-Structured Directories

- Path names can be of two types: absolute and relative. In UNIX and Linux, an **absolute path name** begins at the root and follows a path down to the specified file

- A **relative path name** defines a path from the current directory

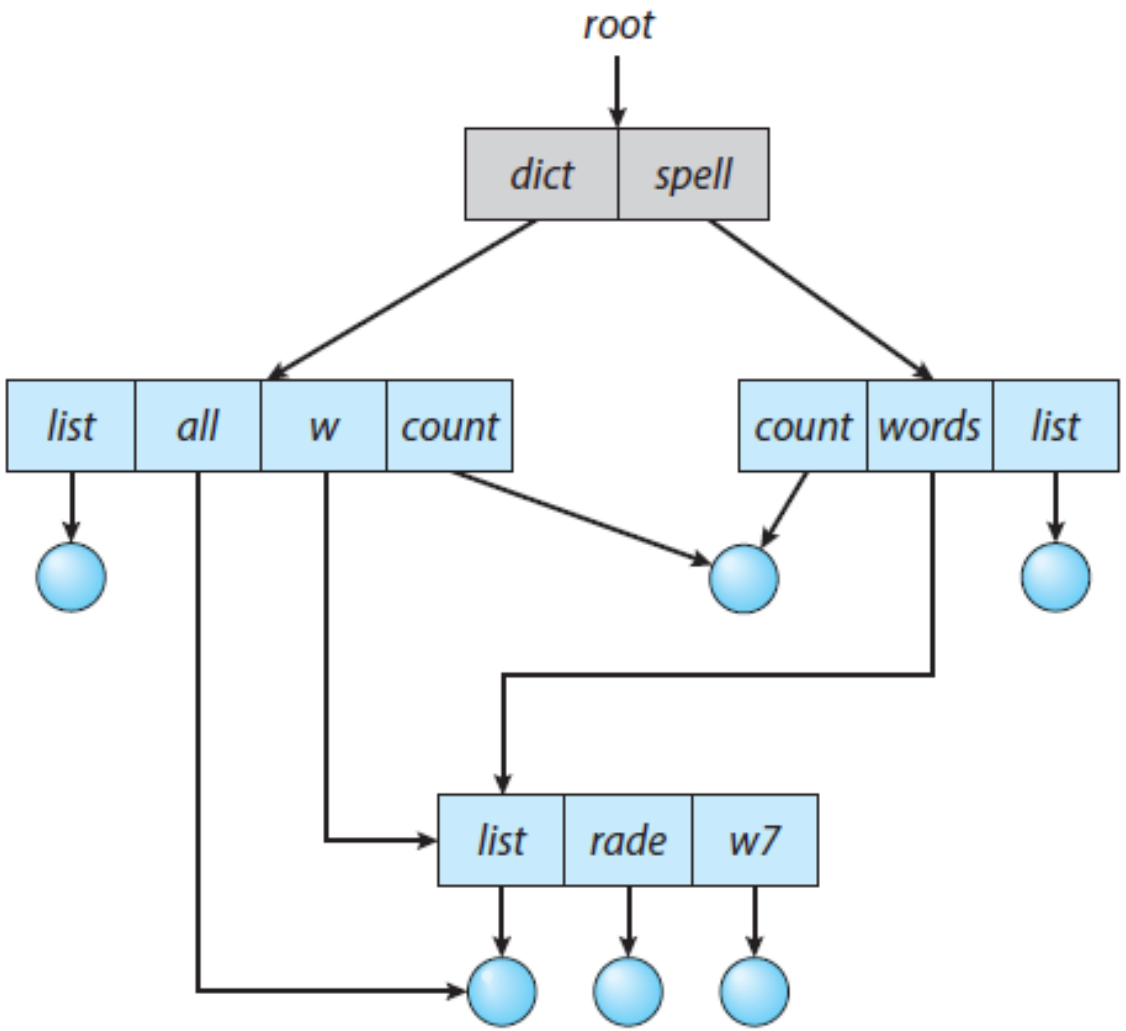
- Allowing a user to define her own subdirectories permits her to impose a structure on her files

- Two approaches to delete a directory: (1) the user must first delete all the files in that directory (or) (2) use rm command

Directory Structure – Acyclic-Graph Directories

- A tree structure prohibits the sharing of files or directories.
- An acyclic graph— a graph with no cycles—allows directories to share subdirectories and files
- The same file or subdirectory may be in two different directories.
- With a shared file, only one actual file exists, so any changes made by one person are immediately visible to the other.

Directory Structure – Acyclic-Graph Directories



Directory Structure – Acyclic-Graph Directories

Shared files and subdirectories can be implemented in several ways:

A common way is to create a new directory entry called a link.

A link is effectively a pointer to another file or subdirectory. For example, a link may be implemented as an absolute or a relative path name.

Another common approach to implement shared files is simply to duplicate all information about them in both sharing directories

When can the space allocated to a shared file be deallocated and reused?

Directory Structure – General Graph Directories

A serious problem with using an acyclic-graph structure is ensuring that there are no cycles.

Adding links to the tree structure resulting in a simple graph structure

The primary advantage of an acyclic graph is the relative simplicity of the algorithms to traverse the graph and to determine when there are no more references to a file.

It avoids traversing shared sections of an acyclic graph twice (while searching) – improves performance.
Avoid cycle otherwise repetitive search – degrades performance

Directory Structure – General Graph Directories